High-Level Structure of the ECDSAfail Frontier Point-Addition Circuit

Technical note on the main quantum frontier

22 July 2026

Abstract

We describe the point-addition circuit accepted in commit `3f8ce09` of the ECDSAfail challenge. The circuit adds a classical SECP256K1 point to a quantum point with an average of 1,318,328 executed Toffoli gates and a peak width of 1,152 qubits, giving the current main-frontier score 1,518,713,856. Its architecture combines an in-place affine state machine, a compressed-history binary extended Euclidean algorithm, half-size symmetric squaring, cleanup using measurements, and a sequence of post-construction circuit rewrites. The presentation follows the high-level, circuit-oriented organization of Section 2 of Schrottenloher [1], while the text and analysis are specific to the challenge implementation.

1 High-Level Structure

Let p be the 256-bit prime

$$p = 2^{256} - 2^{32} - 977, \tag{1}$$

and let $E(\mathbb{F}_p)$ denote the group of points on the SECP256K1 curve

$$E: \quad y^2 = x^3 + 7 \pmod{p}. \tag{2}$$

The challenge interface supplies a point $P = (P_x, P_y)$ in two 256-qubit registers and a runtime-classical point $Q = (Q_x, Q_y)$ in two classical 256-bit registers. The latter is not hard-coded into the emitted operation stream: the validator changes Q between shots, but it never places Q in quantum superposition. For $R = P + Q$, the desired transformation is

$$U_Q: \quad |P_x, P_y\rangle |Q_x, Q_y\rangle_{\mathrm{cl}} \longmapsto |R_x, R_y\rangle |Q_x, Q_y\rangle_{\mathrm{cl}}. \tag{3}$$

The classical registers control gates but do not add 512 qubits to the quantum width. Only the two coordinate registers and live arithmetic workspace enter the peak-qubit count.

Main frontier. The circuit considered here is the accepted submission `830ebf3e`, represented by commit `3f8ce09` on the repository's main branch [5]. As observed on 22 July 2026, the public dashboard reports

$$T_{\mathrm{avg}} = 1{,}318{,}328, \qquad N_Q = 1{,}152, \qquad T_{\mathrm{avg}} N_Q = 1{,}518{,}713{,}856. \tag{4}$$

This is the normal product-score frontier, rather than the separate ranking that minimizes qubits without requiring promotion [6].

Quantum circuits. The emitted circuit uses X, CX, CCX, swaps, and diagonal phase operations, together with measurement/reset cleanup denoted HMR and classical feed-forward. The score counts the average number of executed Toffoli gates over the challenge shots and multiplies it by the maximum number of simultaneously live qubits. Correctness additionally requires the output coordinates to agree with the classical group law, all non-output qubits to return to zero, the accumulated relative phase to vanish, and the forward-then-reverse circuit to restore its input [7].

Point addition. For the generic affine case define

$$\begin{aligned} d_x &= P_x - Q_x, & d_y &= P_y - Q_y, \\ \lambda &= d_y d_x^{-1} \pmod p. \end{aligned} \tag{5}$$

Changing both signs leaves the usual affine slope unchanged. The output coordinates are

$$R_x = \lambda^2 - P_x - Q_x, \qquad R_y = \lambda(Q_x - R_x) - Q_y. \tag{6}$$

The key to an in-place reversible implementation is that the slope is determined both by the input-side difference and by the result point:

$$\lambda = \frac{P_y - Q_y}{P_x - Q_x} = -\frac{R_y + Q_y}{R_x - Q_x} = \frac{R_y + Q_y}{Q_x - R_x} \pmod p. \tag{7}$$

The second expression is the slope of the reverse addition $R + (-Q) = P$ and is independent of the overwritten input P. Proos and Zalka introduced this output-side recomputation in their decomposition of the group shift [2, Sec. 4.3]. Roetteler et al. explicitly credit that construction in their in-place point addition and instantiate it in a straight-line reversible algorithm [3, Sec. 4.1].

Adapted to the sign convention of the challenge circuit, the reversible chain is

$$\begin{aligned} (P_x, P_y) &\longleftrightarrow (P_x - Q_x, P_y - Q_y) \longleftrightarrow (P_x - Q_x, \lambda) \\ &\longleftrightarrow (Q_x - R_x, \lambda) \longleftrightarrow (Q_x - R_x, R_y + Q_y) \longleftrightarrow (R_x, R_y). \end{aligned} \tag{8}$$

Each arrow is reversible when $P_x - Q_x$ and $Q_x - R_x$ are nonzero. In particular, the second half does not need a saved copy of P: by (7), multiplication by $Q_x - R_x$ converts the slope into $R_y + Q_y$ and is the reverse of recomputing the quotient from the output. This is why the slope and its arithmetic workspace can be erased without leaving input-dependent garbage.

The apparently unusual addition of $3Q_x$ is the coordinate bookkeeping that realizes the middle arrow in (8). Since the X register already holds $d_x = P_x - Q_x$, the affine formula gives

$$\begin{aligned} Q_x - R_x &= Q_x - (\lambda^2 - P_x - Q_x) \\ &= (P_x - Q_x) + 3Q_x - \lambda^2 = d_x + 3Q_x - \lambda^2. \end{aligned} \tag{9}$$

Hence the two source operations

$$X \leftarrow X + 3Q_x, \qquad X \leftarrow X - \lambda^2 \tag{10}$$

produce $Q_x - R_x$ exactly. Equivalently, in the notation $(x, y) + (\alpha, \beta) = (x', y')$ of Proos and Zalka,

$$x' - \alpha = \lambda^2 - (x - \alpha) - 3\alpha. \tag{11}$$

The factor three is therefore not an unrelated field-arithmetic optimization; it is the algebraic consequence of retaining $x - \alpha$ in place while changing from the input-side to the output-side slope witness. The same $+3x_2$ step appears in Algorithm 1 of Roetteler et al. [3, Sec. 4.1].

Algorithm 1 states the logical computation. Its variables denote field values; the implementation continually recycles physical qubit identities after proving that the former contents are no longer needed.

Algorithm 1 In-place generic addition of the classical point Q

Require: $P = (P_x, P_y) \in E(\mathbb{F}_p)$ in quantum registers
Require: $Q = (Q_x, Q_y) \in E(\mathbb{F}_p)$ in classical registers
Require: $P_x \neq Q_x$ and $Q_x \neq R_x$
Ensure: the quantum coordinate registers contain $R = P + Q$

1: $X \leftarrow P_x - Q_x$; $Y \leftarrow P_y - Q_y$
2: $Y \leftarrow YX^{-1}$ ▷ $Y = \lambda$
3: $X \leftarrow X + 3Q_x$ ▷ $X = P_x + 2Q_x$
4: $X \leftarrow X - Y^2$ ▷ $X = Q_x - R_x$
5: $Y \leftarrow YX$ ▷ $Y = \lambda(Q_x - R_x)$
6: $Y \leftarrow Y - Q_y$ ▷ $Y = R_y$
7: $X \leftarrow Q_x - X$ ▷ $X = R_x$

Source correspondence. This seven-step trace is a direct transcription of `src/point_add/trailmix_ludicrous/ec_add.rs`, lines 287–309. The two quantum and two classical interface registers are allocated in `src/point_add/trailmix_ludicrous/mod.rs`, lines 362–378; the paired GCD maps are defined in `src/point_add/trailmix_ludicrous/gcd.rs`, lines 1447–1485; and the square state machine is in `src/point_add/trailmix_ludicrous/square.rs`, lines 471–510. Thus the figure below is based on the active call path, not only on the affine formulas.

Figure 1 renders the same straight-line program as a register-level circuit. Its compact block notation follows the visual convention used for constant arithmetic by Häner et al. [4, Fig. 2], while its arithmetic ordering should be compared more directly with their affine point-addition circuits in Figures 9 and 10. The third bus is essential for reading the figure correctly: it denotes the allocator's recycled physical workspace, not a third persistent field value. The load, GCD, and SQ boxes show which macro-block owns some part of that pool; their grey dotted links are workspace-use annotations, not circuit-control wires.

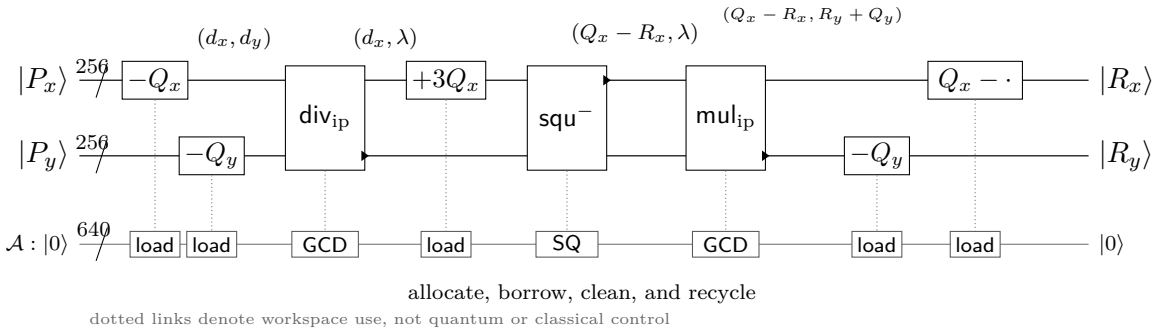

Figure 1: Register and workspace structure of the frontier point-addition circuit. The in-place blocks implement $\mathsf{div}_{\mathrm{ip}} : Y \leftarrow Y/X$, $\mathsf{squ}^- : X \leftarrow X - Y^2$, and $\mathsf{mul}_{\mathrm{ip}} : Y \leftarrow YX$; a black triangle identifies the output leg of a multi-register block. The 640-qubit $\mathcal{A}$ bus is pooled physical workspace whose logical roles change over time, and every non-output qubit is returned to zero. There is deliberately no control, separate slope, address, or point-cache wire.

Exact relation to Figure 9 of Häner et al. Figure 9 in [4] has four persistent logical buses before its temporary lanes: the two point coordinates, a control qubit, and an n-qubit slope register initialized to $|0\rangle$. At the level of field values, its first pair of nonlinear blocks computes the slope out of place and then clears the old numerator,

$$(X, Y, 0)_\Lambda \xrightarrow{\mathsf{div}} (X, Y, Y/X)_\Lambda \xrightarrow{\mathsf{mul}} (X, 0, Y/X)_\Lambda. \tag{12}$$

After the square has formed the output-side difference, the mul^+ block writes the new numerator and the second div erases Λ by recomputing the slope from the output point. Schematically,

$$(X, 0, \lambda)_\Lambda \xrightarrow{\mathsf{mul}^+} (X, \lambda X, \lambda)_\Lambda \xrightarrow{\mathsf{div}} (X, \lambda X, 0)_\Lambda. \tag{13}$$

This is the circuit-level realization of (7).

The frontier code makes a different width–time tradeoff. The function `mod_mul_inverse_in_place` supplies the two bijections

$$D_{\mathrm{ip}}(X, Y) = (X, Y/X), \qquad M_{\mathrm{ip}}(X, Y) = (X, YX). \tag{14}$$

Thus D_{ip} logically fuses the first $\mathsf{div};\mathsf{mul}$ pair of Figure 9, and M_{ip} fuses its $\mathsf{mul}^+;\mathsf{div}$ pair. The visible Y register itself holds λ between the two blocks. This removes Figure 9's persistent n-qubit Λ bus, but it does not make division ancilla-free: each in-place map uses a forward and a reverse extended-GCD walk in the pooled workspace.

The absence of a control qubit also explains the right edge of Figure 1. For the active branch, the last three X-operations of Figure 9 are

$$X \longmapsto X - 2Q_x \longmapsto X - 2Q_x + Q_x \longmapsto -(X - Q_x) = Q_x - X. \tag{15}$$

Häner et al. keep them separate so that the control-off path is the identity. The challenge has no control-off path, so `coord_rsub` implements the composite $X \leftarrow Q_x - X$ directly. Likewise, all classically supplied coordinate updates are unconditional.

Why the cited circuits control the y_2 translations. Both Roetteler et al. and Figure 9 of Häner et al. implement the coherent selection

$$|c\rangle|P_1\rangle \longmapsto |c\rangle|P_1 + [c]P_2\rangle, \qquad c \in \{0, 1\}, \tag{16}$$

needed when a Shor exponent qubit selects a precomputed multiple. Their initial $x_1 \leftarrow x_1 - x_2$ is deliberately unconditional: it supplies the generic nonzero denominator in both branches and is part of a translation frame that is undone on the $c = 0$ branch. In Algorithm 1 of Roetteler et al., controlling $y_1 \leftarrow y_1 - y_2$ selects the real slope $(y_1 - y_2)/(x_1 - x_2)$ when $c = 1$ and a disposable value $y_1/(x_1 - x_2)$ when $c = 0$; the intervening reversible arithmetic clears the latter and the final unconditional $x_1 \leftarrow x_1 + x_2$ closes the inactive branch. In the optimized Figure 9, the divisions themselves are controlled, so $\Lambda = 0$ for $c = 0$ and the unconditional x translations cancel as $-x_2 + 3x_2 - 2x_2 = 0$. In both constructions, however, an unconditional $-y_2$ would leave the inactive Y coordinate translated, so the entrance and exit y_2 corrections are controlled. The frontier challenge kernel has no qubit c: it always performs the addition. Its per-bit `x_if_bit` operations are conditions on runtime-classical `BitId` values used to load Q, not instances of (16).

Why Figure 10 is not the implemented circuit. Figure 10 of [4] is a signed, windowed scalar-multiplication component. It contains an ℓ-qubit address, a sign qubit, repeated ln_i quantum look-ups, and two cache registers holding a point selected in superposition. Its TeX/QPIC source shows three load–use–unload episodes: one for the initial coordinate differences, one for $3Q_x$, and one for the output-side corrections. Repeating the look-up allows the cache lanes to be recycled during the expensive nonlinear blocks.

The frontier circuit contains none of the address, sign, QROM, or quantum point-cache machinery. The input Q is already available in classical `BitId` registers. Consequently, the frontier circuit is a generic classical-point-addition kernel, not a complete implementation of the windowed Shor layer in Figure 10. Replacing Figure 10's quantum cache by the present interface would be invalid when the address is in superposition. The similarity is only in memory discipline: both constructions release coordinate-loading workspace before entering division or multiplication.

Coordinate-loading ancillas. Because Q is runtime-classical, no quantum wire for it persists across the point addition. In the active branch of `coord_addsub`, a clean 256-qubit temporary is allocated, populated with classically conditioned X operations, used as the source of a modular add/subtract, unloaded by the same conditioned X operations, and returned to the free pool. The $+3Q_x$ block first computes $3Q_x \bmod p$ entirely in classical `BitId` workspace and then performs the same 256-qubit load–add– unload pattern. The fused final reverse subtraction also allocates a 256-qubit loaded source. Carry and comparison qubits used inside those modular operations are additional transient members of $\mathcal{A}$.

GCD and square workspace. The two GCD blocks each allocate a 256-qubit temporary and swap the current Y value into it, leaving the visible Y wires zero for the result. A second operand u is initialized to p on as many as 256 qubits. Three branch symbols per one of the 258 jump-2 steps are compressed into a dialog tape. With the accepted configuration, the completed tape occupies

$$2 + 85 \cdot 7 + 5 = 602 \tag{17}$$

qubits: two bits encode step zero, 85 seven-bit words encode triples of steps, and one five-bit word encodes the remaining pair. This 602-qubit tape never coexists with two full-width GCD operands: the schedule frees high operand lanes as the tape grows and consumes the tape while the operands are regrown. Comparator, carry, codec, and branch-flag qubits are allocated only for the current step.

For the square, $\mathcal{A}$ successively contains the 129-bit register $c = a + b$, then one product register at a time: 258 qubits for c^2, or 256 qubits for a^2 and b^2. Reduction and adaptive-adder workspace is borrowed around each application and cleaned before the product is uncomputed. There is no persistent 512-qubit register for λ^2.

Table 1: Logical data and auxiliary storage in the accepted frontier implementation. Sizes are quantum bits unless explicitly marked classical.

Role	Named size	Lifetime and cleanup
Interface X, Y	$256 + 256$	Persist for the whole circuit; X may migrate to recycled physical wires and is routed back at the end.
Loaded coordinate	256	Allocated separately for each Q_x, Q_y, or $3Q_x$ update; unloaded and zeroed immediately after the modular operation.
Saved Y	256	Holds the old numerator or factor while a GCD map writes into the visible Y register; zeroed before return.
GCD operand and dialog	256 and 602	Operand widths shrink as the compressed history grows; reverse replay consumes the tape and restores X.
Square data	129 plus 258	Half-sum plus one partial product; products and the sum are uncomputed in sequence.
Local scratch	variable	Carry, comparison, codec, reduction, and branch lanes; includes clean, measured-and-reset, and temporarily borrowed wires.
Classical storage	not qubits	Q_x, Q_y, the computation of $3Q_x$, conditions, and measurement records are `BitId` values and do not enter quantum width.

Measured peak-liveness audit. Instrumenting the accepted builder with its own phase and allocation census confirms

$$N_Q = 1152 = 512 \text{ interface qubits} + 640 \text{ additional physical qubits.} \tag{18}$$

The first recorded peak occurs in `tlm_apply_inverse_mod_sub_register`. At that instant, the logical ownership census is: 256 visible Y wires, 256 saved-Y wires, 255 X-operand lanes, 255 u-operand lanes, 126 adaptive-adder lanes, one modular-subtraction carry, and three GCD flags, totaling 1,152. The two 255-lane counts arise because known lanes have temporarily been loaned; a logical-role census need not respect the fixed partition between interface and auxiliary physical IDs.

The square independently reaches the same cap. One representative peak has 512 coordinate roles, the 129-bit half-sum, a 256-bit a^2 product, a 128-bit reduction temporary, 125 adaptive-adder lanes, and two carry/control lanes. Hence 1,152 is not merely a declared target: both nonlinear engines actually saturate it. Every temporary is nevertheless zero or measurement-reset with its phase correction before the circuit returns, as required by the challenge's ancilla and phase-garbage checks.

The coordinate arithmetic is specialized to the pseudo-Mersenne identity

$$2^{256} \equiv F \pmod{p}, \qquad F = 2^{32} + 977. \tag{19}$$

Overflow can consequently be folded with a sparse constant. The adders use ripple-carry structures, and their carry workspace is scheduled or borrowed from lanes whose values are restored before the routine returns.

Inverse and forward multiplication. The dominant arithmetic component provides two in-place maps,

$$M_{\text{inv}}: \quad |X, Y\rangle \longmapsto |X, YX^{-1}\rangle, \tag{20}$$

$$M_{\text{fwd}}: \quad |X, Y\rangle \longmapsto |X, YX\rangle. \tag{21}$$

Both use the same 258-iteration, jump-2 binary extended Euclidean schedule. Conceptually, the circuit initializes two GCD operands with p and X, removes powers of two, compares and conditionally exchanges the operands, subtracts the smaller from the larger, and applies the corresponding transformation to a coefficient register. Since every nonzero element of $\mathbb{F}_p$ is invertible, the terminal GCD is one.

A classical binary-GCD program discards its branch decisions. A reversible implementation must retain enough information to replay them. The circuit therefore records a dialog containing shift, subtraction, and swap symbols, but compresses legal windows rather than storing the raw transcript. During the reverse walk, only the next required window is decompressed and then freed. Operand, comparator, and carry widths shrink according to static schedules as their upper lanes become known zero.

For M_{inv}, the coefficient transformation is applied during the forward GCD walk, after which the GCD and compressed dialog are reversed. For M_{fwd}, the forward walk first constructs the branch dialog; the multiplication is produced while that walk is replayed backward. Thus the inverse and product share the same expensive reversible control structure, differing primarily in the direction in which coefficient updates are attached.

Modular square. The square block subtracts λ^2 directly from the X accumulator without retaining a generic 512-bit product. Write

$$\lambda = a + 2^{128}b, \qquad c = a + b. \tag{22}$$

Since $c^2 = a^2 + 2ab + b^2$, the square can be rearranged as

$$-\lambda^2 \equiv -a^2 - 2^{128}c^2 + 2^{128}a^2 + 2^{128}b^2 - Fb^2 \pmod{p}. \tag{23}$$

The circuit constructs and erases the partial squares c^2, a^2, and b^2 one at a time. Diagonal terms are copies of input bits; off-diagonal terms are reversible ANDs counted twice at the appropriate shift. The sparse factor F in (19) becomes a short signed sum of shifts. Only one partial product is live at a time, which trades additional additions for a lower peak width.

Measurement-assisted cleanup and routing. Temporary nonlinear products may be removed by measuring an AND ancilla in the X basis and resetting it. This can leave a branch-dependent sign, so a classically conditioned CZ correction restores coherence. The criterion for freeing a qubit is therefore both a zero value and a clean relative phase.

The allocator returns zeroed temporaries to a free pool and allows selected known-zero or dirty lanes to be loaned to nested arithmetic. Because the GCD operands shrink and regrow, the logical X register need not remain on its original ordered set of physical wires. A final permutation of swaps routes R_x back to the 256 wires declared by the external interface.

Circuit rewriting. After the arithmetic circuit is built, the operation stream is transformed by several semantics-preserving passes. Constant and affine propagation removes gates whose controls

are known, lowers Toffolis when a control is fixed, and cancels inverse pairs. A fanout conjugation rewrites

$$\mathrm{CCX}_{a,b\to t}\,\mathrm{CX}_{t\to u}\,\mathrm{CCX}_{a,b\to t} = \mathrm{CX}_{t\to u}\,\mathrm{CCX}_{a,b\to u}, \tag{24}$$

saving one Toffoli while preserving the value of t.

The last commit adds a straddle-aware cancellation for pairs of CCZ gates. If the unordered support and classical condition context agree, and static write-history analysis proves that the three supported bit values are restored between the gates, then

$$\mathrm{CCZ}\,V\,\mathrm{CCZ} = V. \tag{25}$$

Unlike an adjacency peephole, this rule permits intervening gates whose net effect on the support is the identity. A final CCX cancellation hook is also present, although it is disabled unless its feature variable is enabled.

The emitted stream then passes through a D2 table that removes 1,999 absolute operation indices identified as non-firing under a large input census. Finally, 48 pairs of X gates encode a nonce:

$$(X_q X_q)^{48} = I. \tag{26}$$

This tail is exactly the identity as a quantum operation, but it changes the Fiat–Shamir-derived validation samples.

Table 2: Main-frontier resource metrics for commit `3f8ce09`.

Metric	Value
Average executed Toffoli gates	1,318,328
Peak live qubits	1,152
Product score	1,518,713,856
Dashboard status	Promoted frontier leader

Correctness scope. The arithmetic state trace, pseudo-Mersenne reductions, reversible GCD replay, fanout conjugation, and restored-support CCZ cancellation admit algebraic explanations. The D2 table has a different assurance level: the checked-in header reports a census over 10^8 generated inputs, while the application pass removes gates by absolute index rather than reproving their inactivity. The two-bit narrowing of early GCD comparisons is likewise motivated by observed slack in the commit rather than by a machine-checked universal bound. These benchmark specializations should be distinguished from structural circuit identities.

The routine also implements only generic affine addition. The validator excludes the point at infinity and inputs with $P_x = Q_x$, so this path does not separately handle point doubling, inverse-point addition, or infinity. Subject to that interface, the circuit is best viewed in three layers:

$$\underbrace{\text{affine state machine}}_{\text{field values}} \longrightarrow \underbrace{\text{reversible arithmetic}}_{\text{GCD, square, cleanup}} \longrightarrow \underbrace{\text{stream rewriting}}_{\text{propagation and cancellation}}. \tag{27}$$

The resulting frontier point is not obtained by minimizing width or Toffoli count in isolation, but by balancing both quantities in the challenge score of (4).

References

[1] A. Schrottenloher, *Optimized Point Addition Circuits for Elliptic Curve Discrete Logarithms*, arXiv:2606.02235, 2026. https://arxiv.org/abs/2606.02235.

[2] J. Proos and C. Zalka, *Shor's Discrete Logarithm Quantum Algorithm for Elliptic Curves*, Quantum Information and Computation, vol. 3, no. 4, pp. 317–344, 2003. https://arxiv.org/abs/quant-ph/0301141.

[3] M. Roetteler, M. Naehrig, K. M. Svore, and K. Lauter, *Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms*, arXiv:1706.06752, 2017. https://arxiv.org/abs/1706.06752.

[4] T. Häner, S. Jaques, M. Naehrig, M. Roetteler, and M. Soeken, *Improved Quantum Circuits for Elliptic Curve Discrete Logarithms*, arXiv:2001.09580, 2020. https://arxiv.org/abs/2001.09580.

[5] ECDSAfail Challenge, *Accepted submission commit `3f8ce09`*, 2026. https://github.com/ecdsafail/ecdsafail-challenge/commit/3f8ce0984f57e87c4ba341556373efb8283ed768.

[6] ECDSAfail Challenge, *Public main-frontier dashboard*, accessed 22 July 2026. https://www.ecdsafail.rocks/.

[7] ECDSAfail Challenge, *Challenge contract, evaluation, and scoring.* https://github.com/ecdsafail/ecdsafail-challenge.